

L07 · Γραφήματα II

Διάσχιση — BFS & DFS

Algorithms Class Hub

Αλγόριθμοι και Πολυπλοκότητα (K17) · ΕΚΠΑ

Οι δύο θεμελιώδεις αλγόριθμοι διάσχισης γραφήματος: αναζήτηση κατά βάθος (DFS) και αναζήτηση κατά πλάτος (BFS). Λύνουν $s-t$ συνεκτικότητα και $s-t$ απόσταση σε χρόνο $O(n + m)$.

1 Τι θα δούμε

Από το προηγούμενο μάθημα ξέρουμε τι είναι ένα γράφημα και πώς το αποθηκεύουμε. Τώρα ήρθε η ώρα για **αλγορίθμους**. Σε αυτή τη διάλεξη θα δούμε τις **δύο θεμελιώδεις τεχνικές διάσχισης**:

1. **DFS (Depth-First Search)** — αναζήτηση κατά βάθος. Πας όσο πιο μακριά μπορείς, μετά γυρνάς πίσω και δοκιμάζεις άλλο δρόμο.
2. **BFS (Breadth-First Search)** — αναζήτηση κατά πλάτος. Εξερευνείς **επίπεδα** — όλους τους γείτονες πρώτα, μετά τους γείτονες των γειτόνων, κ.ο.κ.

Και οι δύο τρέχουν σε χρόνο $O(n + m)$ με λίστα γειτνίασης. Λύνουν την **$s-t$ συνεκτικότητα**. Επιπλέον το BFS υπολογίζει τη **συντομότερη απόσταση** σε γραφήματα **χωρίς βάρη** (κάθε ακμή μετράει 1).

Κλειδί

Από τις σημαντικότερες διαλέξεις του μαθήματος. Σχεδόν κάθε εξεταζόμενο θέμα σε γραφήματα ξεκινάει με «τρέξε BFS» ή «τρέξε DFS» — οι ίδιοι αλγόριθμοι ή παραλλαγές τους είναι τα δομικά μπλοκ για τοπολογική ταξινόμηση, διμερότητα, SCC, και πολλά άλλα που θα δούμε στα επόμενα μαθήματα.

2 DFS — Αναζήτηση κατά βάθος

2.1 Διαισθητική ιδέα

Φαντάσου ότι είσαι στον κόμβο s ενός λαβύρινθου. Διαλέγεις **μία** κατεύθυνση και πας. Φτάνεις σε νέο κόμβο, διαλέγεις **μία** κατεύθυνση που δεν ξέρεις ακόμα, πας. Συνεχίζεις μέχρι να φτάσεις σε αδιέξοδο ή σε κόμβο που έχεις ξαναδεί. Τότε **γυρνάς πίσω** στον προηγούμενο κόμβο και δοκιμάζεις άλλη κατεύθυνση.

Με μία λέξη: βυθίσου όσο πιο βαθιά μπορείς, μετά γύρνα πίσω.

2.2 Γενικός σκελετός — η συνεκτική συνιστώσα

Στις διαφάνειες, το «DFS» στην πρώτη του εκδοχή είναι ένας **γενικός αλγόριθμος εύρεσης συνεκτικής συνιστώσας** — ο **R-set algorithm**:

```

DFS(G, s):
  R = {s}
  ενόσω υπάρχει ακμή {u, v} με u ∈ R και v ∉ R:
    R ← R ∪ {v}
  επίστρεψε R

```

Θεώρημα. Με τον τερματισμό, το R είναι η **συνεκτική συνιστώσα** που περιέχει την s .

Διαίσθητικά: ξεκινάμε από το $\{s\}$. Όσο βρίσκουμε κάποια ακμή που έχει το ένα άκρο μέσα στο R και το άλλο έξω, την «τραβάμε μέσα». Όταν δεν υπάρχει τέτοια ακμή, το R είναι ακριβώς το σύνολο των κορυφών που μπορούμε να φτάσουμε από την s .

Διαίσθηση

Αυτό **δεν** είναι ο κανονικός αναδρομικός DFS, είναι μια γενική περιγραφή. Ο πραγματικός DFS διαλέγει συγκεκριμένη σειρά (βάθος πρώτα) και υλοποιείται είτε αναδρομικά είτε με στοίβα. Αλλά η αλήθεια είναι: **όποια σειρά κι αν διαλέξεις** για την επόμενη ακμή, καταλήγεις στο ίδιο R .

2.3 Αναδρομική υλοποίηση

Η συνηθισμένη εκδοχή του DFS, με χρήση αναδρομής:

```

DFS(G, s):
  R = ∅
  Για κάθε u ∈ V:
    Εξερευνημένη(u) = false
  Εξερευνημένη(s) = true
  R = {s}
  Για κάθε ακμή {s, u} στο G:
    Εάν Εξερευνημένη(u) = false:
      DFS(G, u)      // αναδρομική κλήση

```

Πώς λειτουργεί:

1. Σημαδεύουμε κάθε κορυφή ως «μη εξερευνημένη» αρχικά.
2. Σημαδεύουμε την s ως εξερευνημένη.
3. Για **κάθε** γειτονική κορυφή της s που δεν έχουμε ξανα-επισκεφθεί, **κατεβαίνουμε** εκεί (αναδρομική κλήση).

Παράδειγμα τρεξίματος στο γράφημα του προηγούμενου μαθήματος, ξεκινώντας από το 1:

- Σημαδεύω 1. Γείτονες του 1: $\{2, 3\}$.
- Πάω στο 2. Γείτονες του 2: $\{1, 3, 4, 5\}$ — το 1 εξερευνημένο.
- Πάω στο 3. Γείτονες του 3: $\{1, 2, 5, 7, 8\}$ — 1, 2 εξερευνημένα.
- Πάω στο 5. Γείτονες του 5: $\{2, 3, 4, 6\}$ — 2, 3 εξερευνημένα.
- Πάω στο 4, μετά στο 6 (αδιέξοδα) — πίσω. Πάω στο 7, μετά στο 8.
- Όλη η συνιστώσα εξερευνήθηκε.

Σειρά εξερεύνησης: $1 \rightarrow 2 \rightarrow 3 \rightarrow 5 \rightarrow 4 \rightarrow 6 \rightarrow 7 \rightarrow 8$.

2.4 Πολυπλοκότητα DFS: $O(n + m)$

Με λίστα γειτνίασης: κάθε κορυφή σημαδεύεται **μία φορά** (σύνολο $O(n)$), και κάθε ακμή εξετάζεται **δύο φορές** (μία από κάθε άκρο, σύνολο $O(m)$). Συνολικά $O(n + m)$.

3 BFS — Αναζήτηση κατά πλάτος

3.1 Διαισθητική ιδέα

Φαντάσου τώρα ότι είσαι **κύμα** που εκπέμπεται από την s . Το κύμα εξαπλώνεται **σε όλες τις κατευθύνσεις ταυτόχρονα**. Σε χρόνο 1, φτάνεις στους άμεσους γείτονες. Σε χρόνο 2, στους γείτονες των γειτόνων. Κ.ο.κ.

Με μία λέξη: εξερεύνηση **επίπεδο-επίπεδο**.

3.2 Επίσημος ορισμός

Από την s δημιουργούμε **επίπεδα**:

- $L_0 = \{s\}$.
- $L_1 =$ όλοι οι γείτονες του L_0 που δεν είναι στο L_0 .
- $L_2 =$ όλες οι κορυφές με ακμή προς L_1 , που δεν είναι σε L_0 ή L_1 .
- Γενικά: $L_{i+1} =$ όλες οι κορυφές με ακμή προς L_i , που δεν είναι σε προηγούμενο επίπεδο.

Κλειδί

Θεώρημα. Για κάθε i , το L_i περιέχει **ακριβώς** τις κορυφές σε **απόσταση** i από την s (όπου απόσταση = αριθμός ακμών συντομότερης διαδρομής).

Διαισθητικά: μια κορυφή σε απόσταση 3 δεν μπορεί να εμφανιστεί νωρίτερα από το L_3 , αλλά εμφανίζεται **εξάπαντος** στο L_3 (αφού έχει διαδρομή μήκους 3). Η απόδειξη με επαγωγή στα επίπεδα είναι άμεση.

Συνέπεια: το BFS λύνει το πρόβλημα **s - t απόσταση** σε γραφήματα χωρίς βάρη: η απόσταση είναι το επίπεδο όπου εμφανίζεται η t . Υπάρχει διαδρομή από s σε t αν και μόνο αν η t εμφανίζεται σε **κάποιο** επίπεδο.

3.3 Σημαντική ιδιότητα του BFS-δένδρου

Κλειδί

Λήμμα. Έστω T το BFS-δένδρο που παράγεται ξεκινώντας από την s . Αν $\{x, y\}$ ακμή του G , τότε τα **επίπεδα** των x και y διαφέρουν **το πολύ κατά 1**.

Διαισθητικά: αν είχαμε ακμή που πηδάει 2 επίπεδα, θα έπρεπε ο γείτονας του χαμηλότερου επιπέδου να είχε ήδη ανακαλυφθεί ένα επίπεδο νωρίτερα — αντίφαση.

Συνέπεια: όλες οι ακμές του γραφήματος είτε είναι ακμές **του δέντρου** (μεταξύ διαδοχικών επιπέδων), είτε είναι ακμές **μέσα στο ίδιο επίπεδο**. Καμία ακμή δεν πηδάει επίπεδα. Αυτή η ιδιότητα είναι κλειδί για τη διμερότητα, που θα δούμε παρακάτω.

3.4 Ο αλγόριθμος BFS (αναλυτικά)

```
BFS(G, s):
  Για κάθε  $v \in V$ :
    Αναγνωσμένη( $v$ ) = false
     $\pi[v] \leftarrow \emptyset$ 
  Αναγνωσμένη( $s$ ) = true
   $L[0] = \{s\}$ 
   $i \leftarrow 0$  // δένδρο  $T = \{s\}$ 
  ενόσω  $L[i] \neq \emptyset$ :
    αρχικοποίηση κενής λίστας  $L[i+1]$ 
    για κάθε  $u \in L[i]$ :
      για κάθε ακμή  $\{u, v\} \in E$ :
        εάν Αναγνωσμένη( $v$ ) = false:
          Αναγνωσμένη( $v$ ) = true
           $\pi[v] \leftarrow u$  //  $T = T \cup \{u, v\}$ 
           $L[i+1] \leftarrow L[i+1] \cup \{v\}$ 
     $i \leftarrow i + 1$ 
```

Τι κρατάει ο αλγόριθμος:

- Αναγνωσμένη(v) — έχει επισκεφθεί η κορυφή.
- $\pi[v]$ — ο **γονέας** της v στο BFS-δένδρο. Με αυτό μπορούμε να ανακατασκευάσουμε τη συντομότερη διαδρομή από s σε κάθε κορυφή.
- $L[i]$ — η λίστα κορυφών στο επίπεδο i .

Παραλλαγή με ουρά (queue): αντί να φυλάμε πίνακα από λίστες, μπορούμε να χρησιμοποιήσουμε **FIFO ουρά**. Βγάζεις την κορυφή από την αρχή, βάζεις τους μη-εξερευνημένους γείτονες στο τέλος. Είναι μαθηματικά **ισοδύναμο**.

3.5 Παράδειγμα στο γράφημα του προηγούμενου μαθήματος

Ξεκινώντας από $s = 1$:

- $L_0 = \{1\}$.
- Γείτονες του 1: $\{2, 3\}$. Άρα $L_1 = \{2, 3\}$.
- Γείτονες του 2: $\{1, 3, 4, 5\}$ (νέοι: 4, 5)· γείτονες του 3: $\{1, 2, 5, 7, 8\}$ (νέοι: 7, 8). Άρα $L_2 = \{4, 5, 7, 8\}$.
- Γείτονες του 5: $\{2, 3, 4, 6\}$ (νέο: 6). Άρα $L_3 = \{6\}$.

Αποστάσεις από το 1:

Κορυφή	1	2	3	4	5	6	7	8
Απόσταση	0	1	1	2	2	3	2	2

3.6 Ανάλυση πολυπλοκότητας

Δείχνουμε ότι ο BFS τρέχει σε $O(n + m)$ με λίστα γειτνίασης.

Αφελής φραγμός $O(n^2)$: το πολύ n λίστες $L[i]$ · κάθε κορυφή ανήκει σε το πολύ μία· για κάθε κορυφή u ο βρόχος «για κάθε ακμή $\{u, v\}$ » κάνει το πολύ n επαναλήψεις· $\Theta(1)$ ανά επανάληψη $\rightarrow O(n^2)$ σύνολο.

Σφιχτότερος φραγμός $O(n + m)$: όταν εξετάζουμε την κορυφή u , σαρώνουμε **ακριβώς** $\deg(u)$ προσκείμενες ακμές (όχι n). Σύνολο εξετάσεων:

$$\sum_{v \in V} (1 + \deg(v)) = n + \sum_v \deg(v) = n + 2m$$

(από το handshaking λήμμα του προηγούμενου μαθήματος). Άρα $O(n + m)$.

Κλειδί

Σημαντική παρατήρηση: η ανάλυση $O(n^2)$ είναι **σωστή** ασυμπτωτικά αλλά **όχι σφιχτή**. Για αραιά γραφήματα ($m = O(n)$) ο σφιχτός φραγμός $O(n + m) = O(n)$ είναι **πολύ καλύτερος**. Πάντα ψάχνε σφιχτό φραγμό σε ανάλυση γραφημάτων.

4 Σύγκριση: BFS vs DFS

	BFS	DFS
Στρατηγική	επίπεδο-επίπεδο	βάθος πρώτα
Δομή δεδομένων	ουρά (FIFO)	στοίβα (LIFO) ή αναδρομή
Λύνει	s-t απόσταση (χωρίς βάρη)	s-t συνεκτικότητα
Πολυπλοκότητα	$O(n + m)$	$O(n + m)$
Δένδρο που παράγει	ίδιο επίπεδο = ίδια απόσταση	μακρές αλυσίδες
Εφαρμογές	shortest paths (unweighted), bipartite test, web crawler	τοπολογική ταξινόμηση, SCC, εύρεση κύκλων

Διαίσθηση

Πότε ποιο;

- Θέλω **απόσταση**: BFS.
- Θέλω απλά «**υπάρχει διαδρομή**»;»: είτε δουλεύει, αλλά DFS συχνά πιο φυσικό.
- Θέλω **εύρεση κύκλου** ή **τοπολογική ταξινόμηση**: DFS.
- Θέλω να φτάσω **όσο πιο μακριά μπορώ**: DFS.
- Θέλω «**ποιες κορυφές απέχουν ακριβώς k βήματα**»;»: BFS.

5 Σχέση BFS / DFS με τα προβλήματα του προηγούμενου μαθήματος

Πρόβλημα	Λύση	Πολυπλοκότητα
s-t συνεκτικότητα	DFS ή BFS — δεξ αν το t ανήκει στο R ή σε κάποιο L_i	$O(n + m)$
s-t απόσταση (χωρίς βάρη)	BFS — απόσταση = επίπεδο	$O(n + m)$
Εύρεση όλων των συνεκτικών συνιστωσών	Επανάλαβε DFS/BFS από κάθε ανεξερεύνητη κορυφή	$O(n + m)$

Τι κρατάμε από αυτή τη διάλεξη

1. **DFS** — βυθίζεσαι όσο μπορείς, μετά γυρνάς. Αναδρομική υλοποίηση. $O(n + m)$.
2. **BFS** — εξερευνάς επίπεδα. $L_0 = \{s\}$, L_{i+1} = νέοι γείτονες του L_i . $O(n + m)$.
3. **Θεώρημα BFS**: L_i = όλες οι κορυφές σε απόσταση i . **Συνέπεια**: BFS λύνει s-t απόσταση σε γραφήματα χωρίς βάρη.
4. **Ιδιότητα BFS-δέντρου**: καμία ακμή δεν πηδάει επίπεδα. Κλειδί για τη διμερότητα.
5. **Σύγκριση**: BFS για απόσταση, DFS για βάθος/κύκλους. Και τα δύο σε $O(n + m)$ με λίστα γειτνίασης.
6. **Σφιχτή ανάλυση**: μην αρκείσαι σε $O(n^2)$ — με handshaking παίρνεις $O(n + m)$, που για αραιά γραφήματα είναι πολύ καλύτερο.